

DA5400: Foundations of Machine Learning

Topic 5: Classification



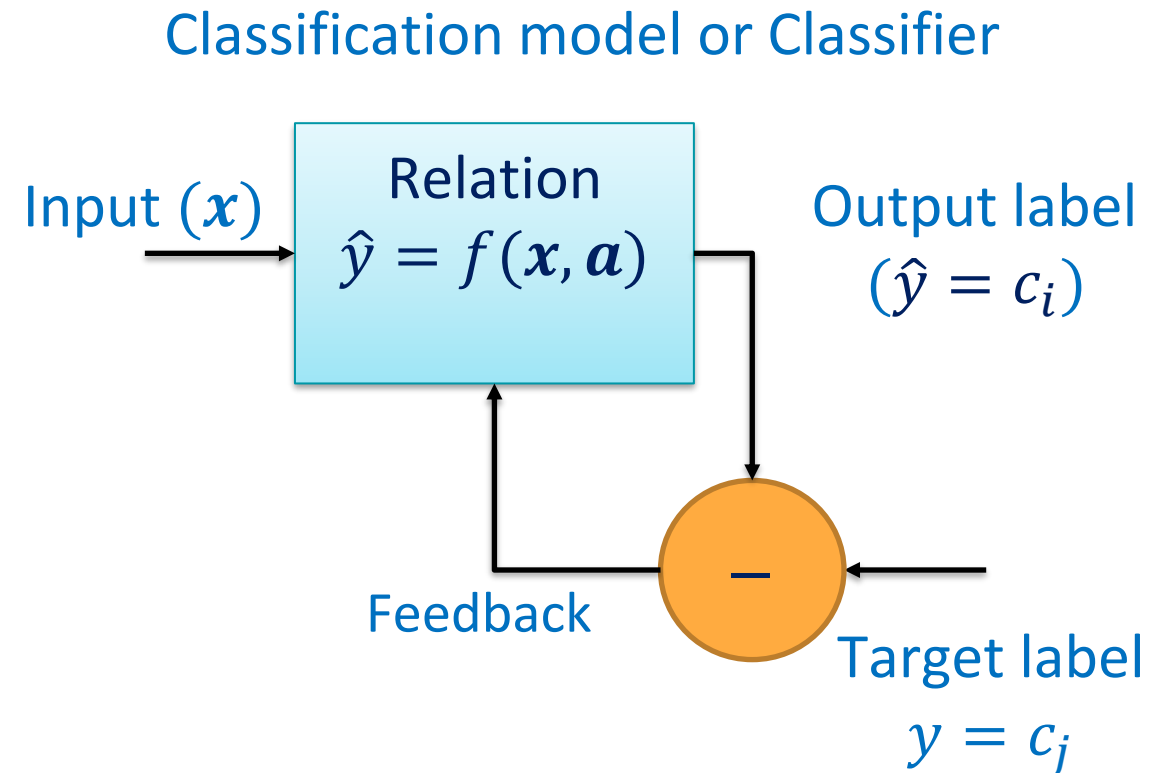
Contents

- Introduction to Classification
- Types of Classification Models
- Classification Algorithms
 - K-Nearest Neighbour classifier
 - Logistic Regression
 - Naïve Bayes Classifier
- Metrics to Evaluate Classification Models

Introduction to Classification

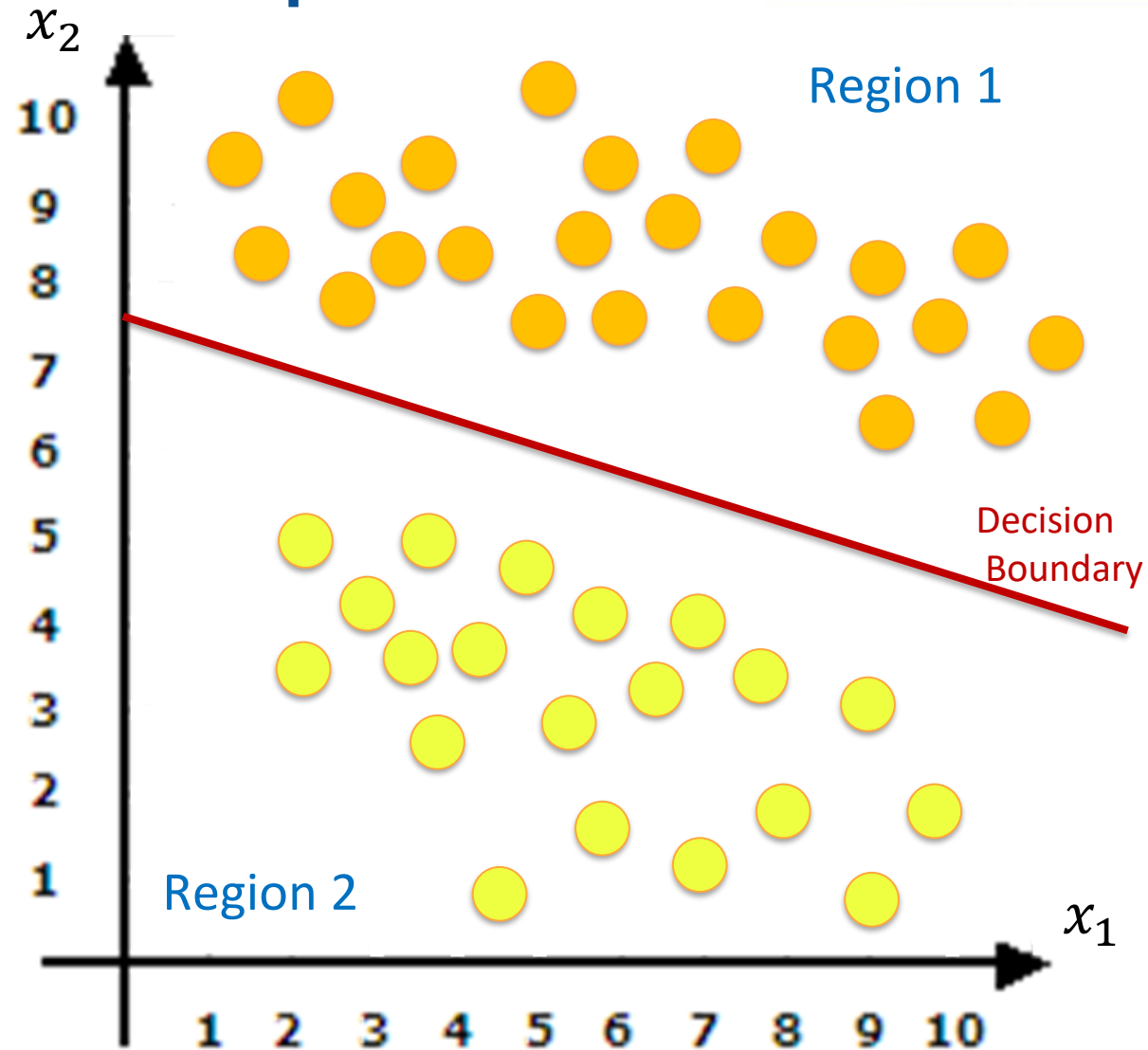
Classification

- A supervised learning task in which the output variables (y) to be predicted are discrete values referred to as class labels
- **Examples:**
 - Classifying emails as spam or not based on subject or body
 - Given handwritten character images, classify it to be one of the known characters
 - Given details of a financial transaction, classifying it to be genuine or fraudulent
 - Given the details of a tumour, classifying it to be benign or malignant
 - Given the parameters of a process in an industry, classifying whether there is fault in the process or not



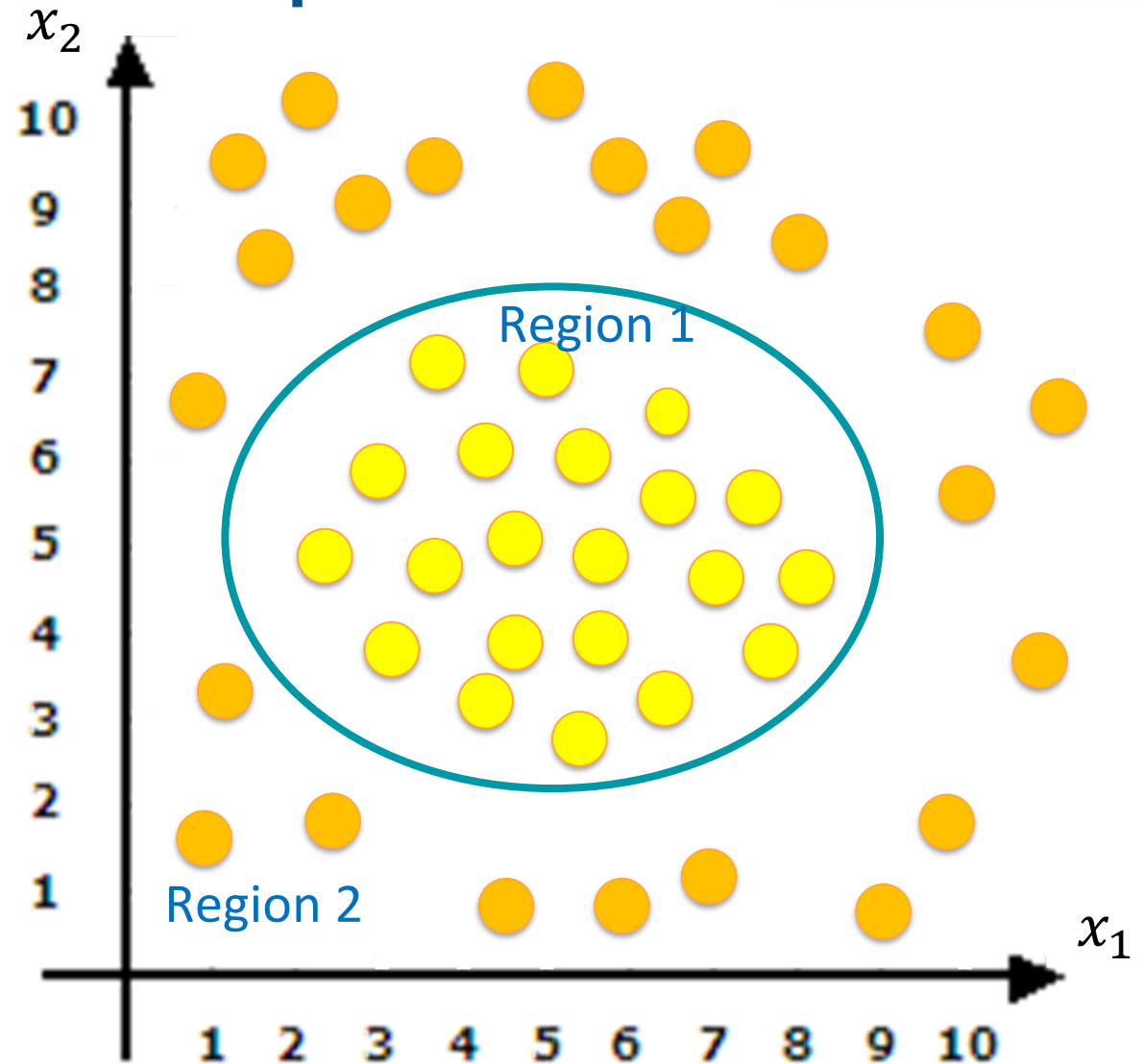
Classification: Feature Space Perspective

- A classifier divides the feature space into a set of regions each having a class label associated with it
- Boundaries between these regions can be straight, curved, smooth or rough depending on the prediction function
- Referred to as decision boundaries



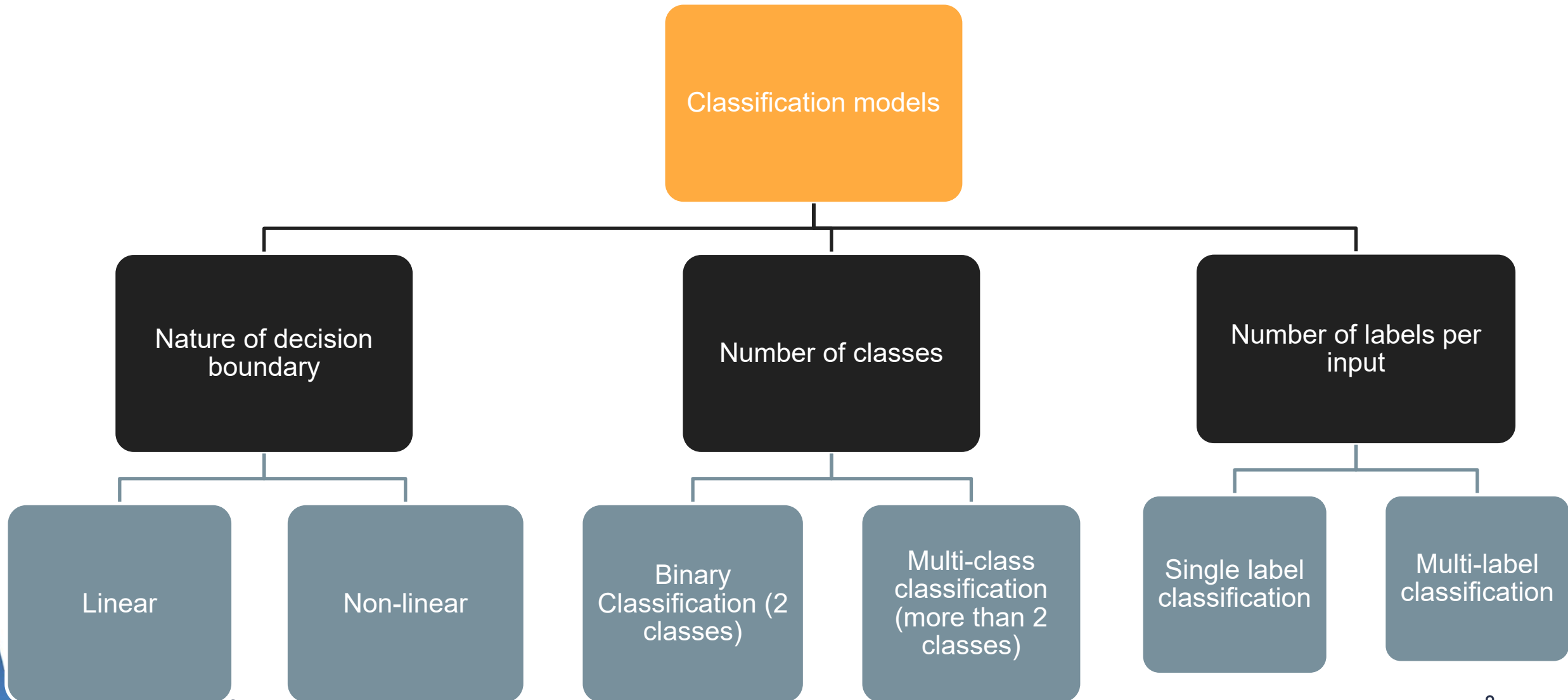
Classification: Feature Space Perspective

- A classifier divides the feature space into a set of regions each having a class label associated with it
- Boundaries between these regions can be straight, curved, smooth or rough depending on the prediction function
- Referred to as **decision boundaries**
- Class predictions will be based on the region in which the new input lies

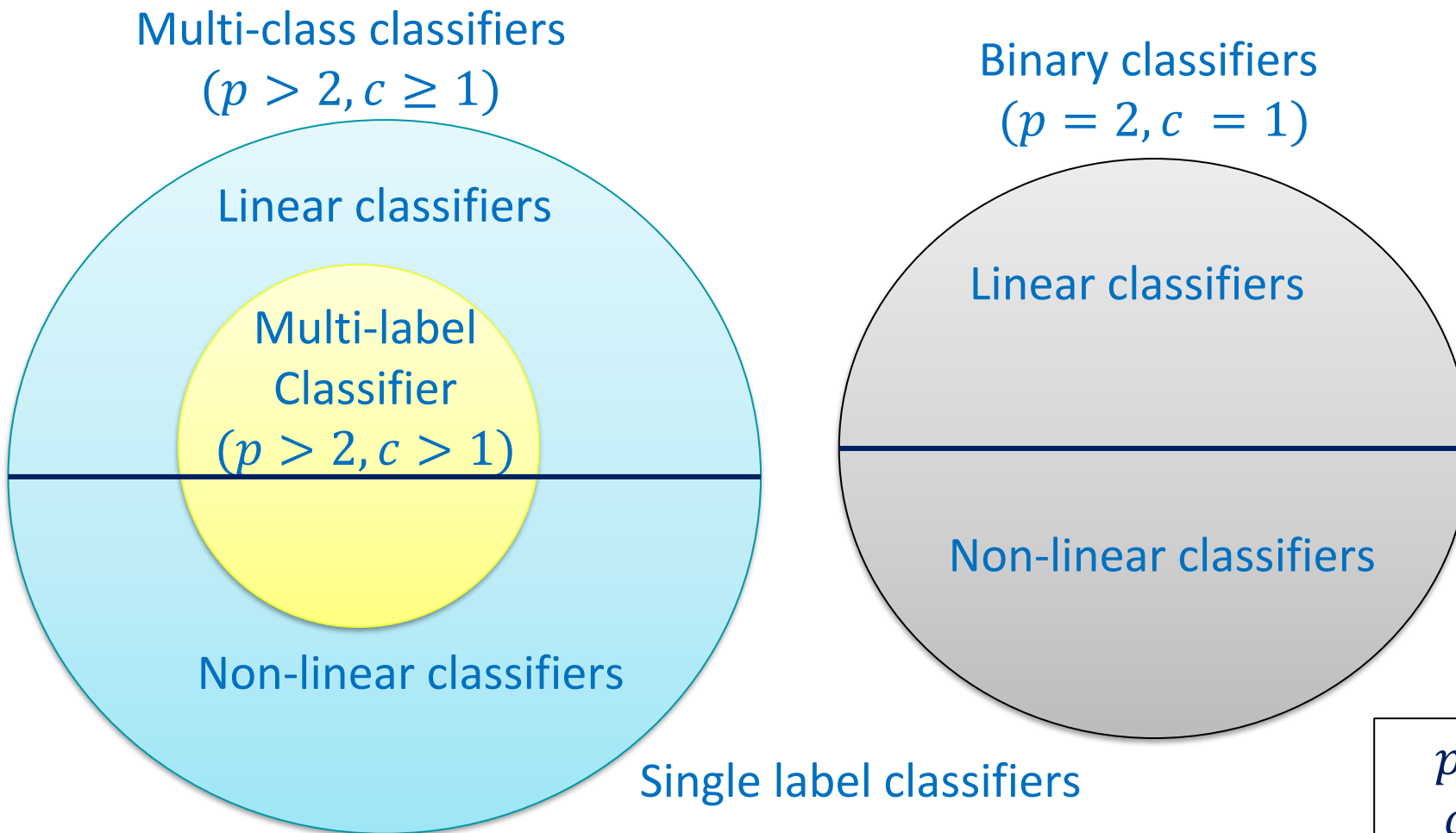


Types of Classification Models

Types of Classification Models



Types of Classification Models



p : Total no of classes
 c : No of class labels
predicted per data point

Soft vs Hard Predictions in Classification

- **Soft Prediction:** Prediction that gives the probability of a data point belonging to each of the target classes
- **Hard Prediction:** Prediction that gives the exact class that the data point might belong to
- Soft prediction gives more information on the confidence of the classifier in the prediction made
- Soft prediction can be turned into a hard prediction by setting a probability threshold

Eg: Predicting whether a transaction is fraud or genuine

p_0 : Probability that transaction is genuine

p_1 : Probability that transaction is fraud

Soft predictions:

Sample 1: $\begin{bmatrix} p_0 \\ p_1 \end{bmatrix} = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix}$

Sample 2: $\begin{bmatrix} p_0 \\ p_1 \end{bmatrix} = \begin{bmatrix} 0.55 \\ 0.45 \end{bmatrix}$

Hard predictions with Threshold = 0.5:

Sample 1 – Genuine

Sample 2 - Genuine

Hard predictions with Threshold = 0.6:

Sample 1 – Genuine

Sample 2 - Fraud

Classification Algorithms

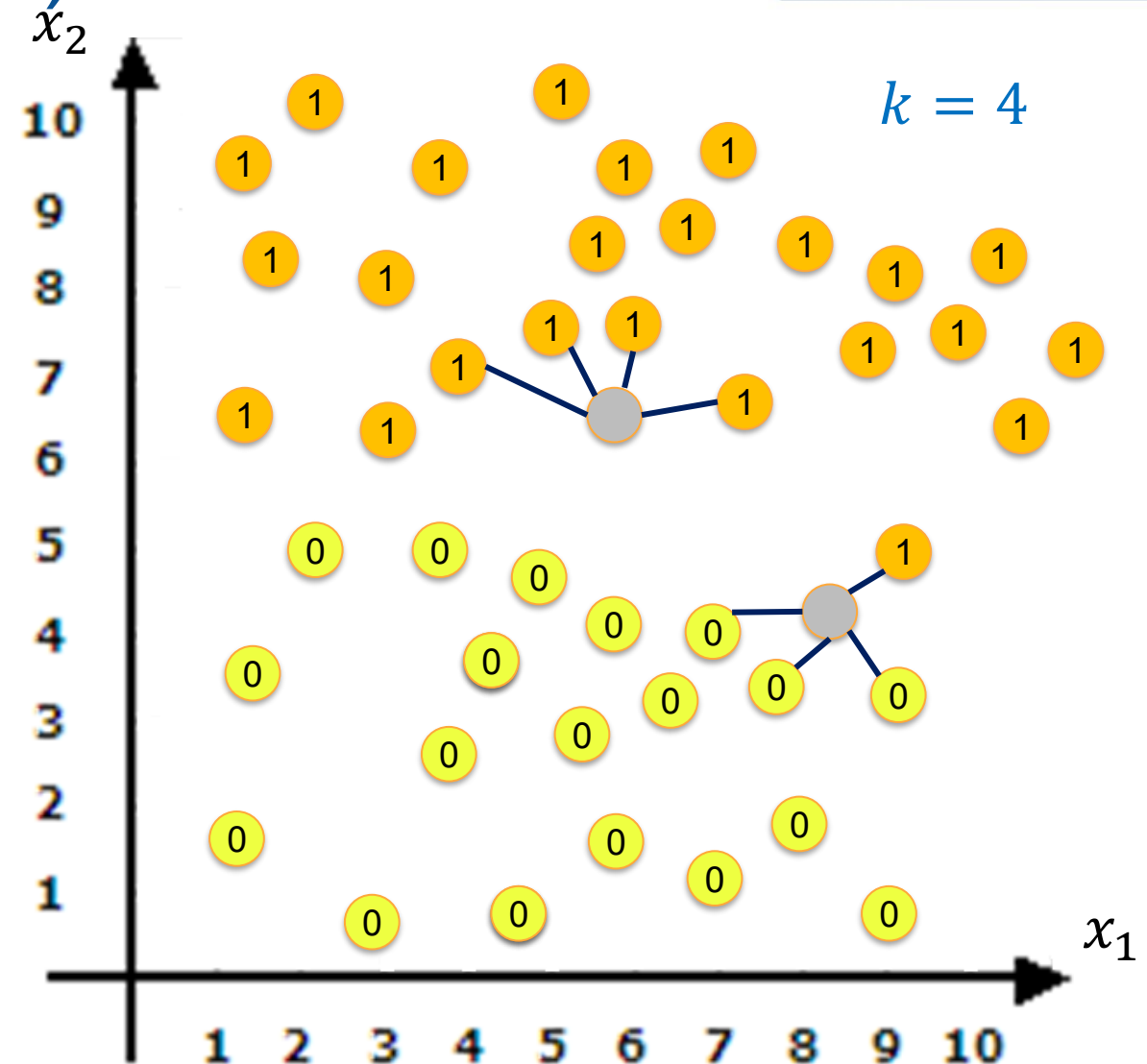
Some Points to Note

- Pre-processing techniques for regression can also be used for classification
- Binary Classification:
 - Classes are referred to as positive (1) and negative (0)
- Model Validation:
 - Model validation techniques used in regression can be used for classification
- Accuracy in prediction:
 - Number of samples correctly predicted by total number of samples used in prediction $\left(\frac{N_c}{N}\right)$
- Error in prediction:
 - Number of samples which are mis-classified i.e., class predicted is different from actual class
 - % error can be taken by dividing the number of mis-classified samples by total number of samples used in prediction $\left(1 - \frac{N_c}{N}\right)$

K-Nearest Neighbour Classifier

k-Nearest Neighbour (kNN) Classifier

- Data points are classified based on their neighbours in the feature space
- Majority of the classes of the k -nearest neighbours is taken as the class for a data point
- k is a hyper-parameter to be decided by the user
- No model is learnt and stored in this classifier – lazy classifier
- It is non-parametric and makes no assumptions on data distribution



k-Nearest Neighbour (kNN) Classifier

- Training data: $\{\mathbf{x}_i, y_i\}_{i=1}^N$
- Distance metric: $d(\mathbf{x}, \mathbf{x}_i)$
- $\mathcal{N}_k(\mathbf{x})$: Indices of k nearest neighbours of $\mathbf{x}$

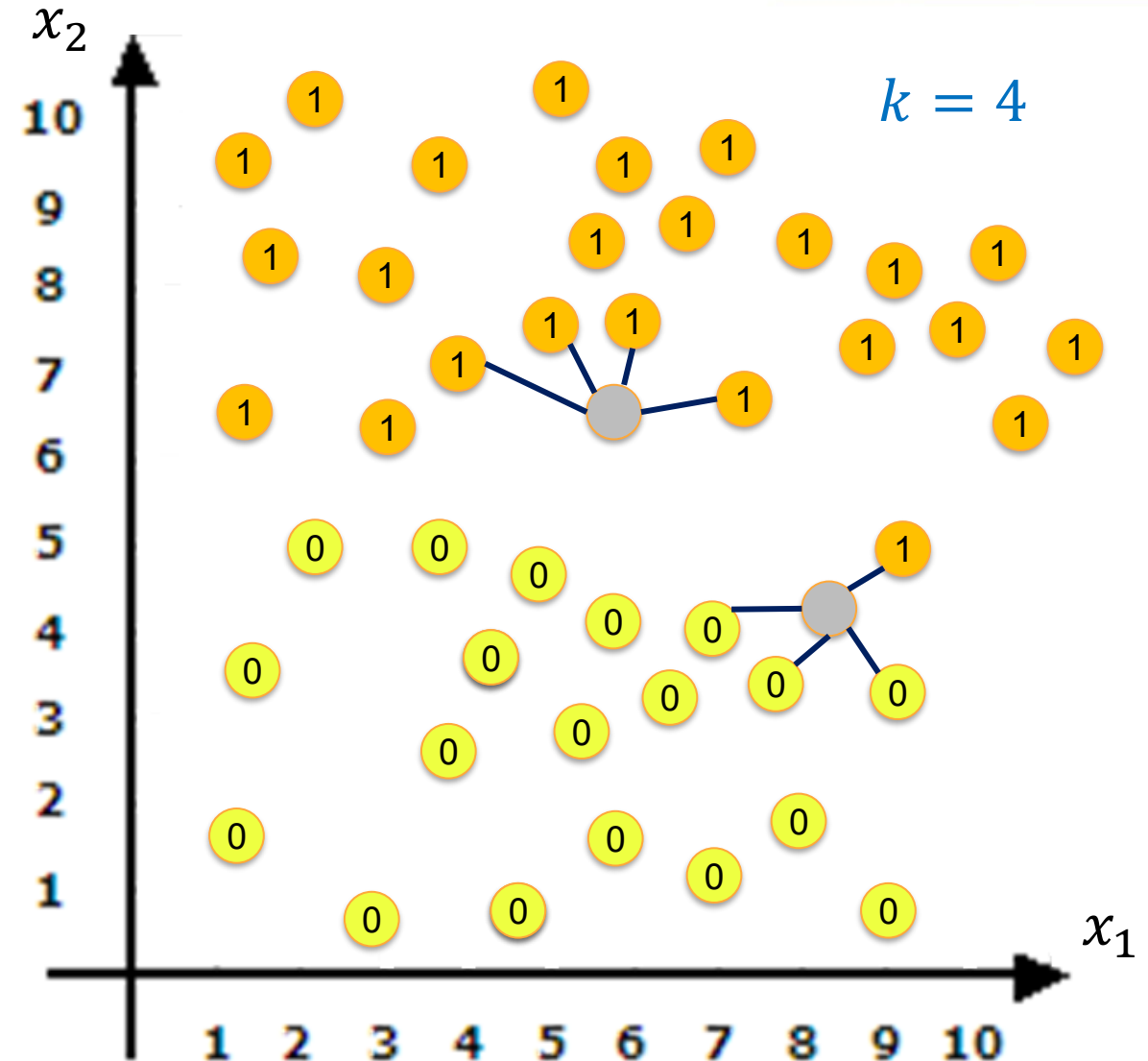
$$\mathcal{N}_k(\mathbf{x}) = \underset{S \subset \{1, 2, \dots, N\}}{\operatorname{argmin}} \sum_{i \in S} d(\mathbf{x}, \mathbf{x}_i)$$

- Hard prediction:

$$\hat{y}_i(\mathbf{x}) = \underset{c}{\operatorname{argmax}} \sum_{i \in \mathcal{N}_k(\mathbf{x})} 1[y_i = c]$$

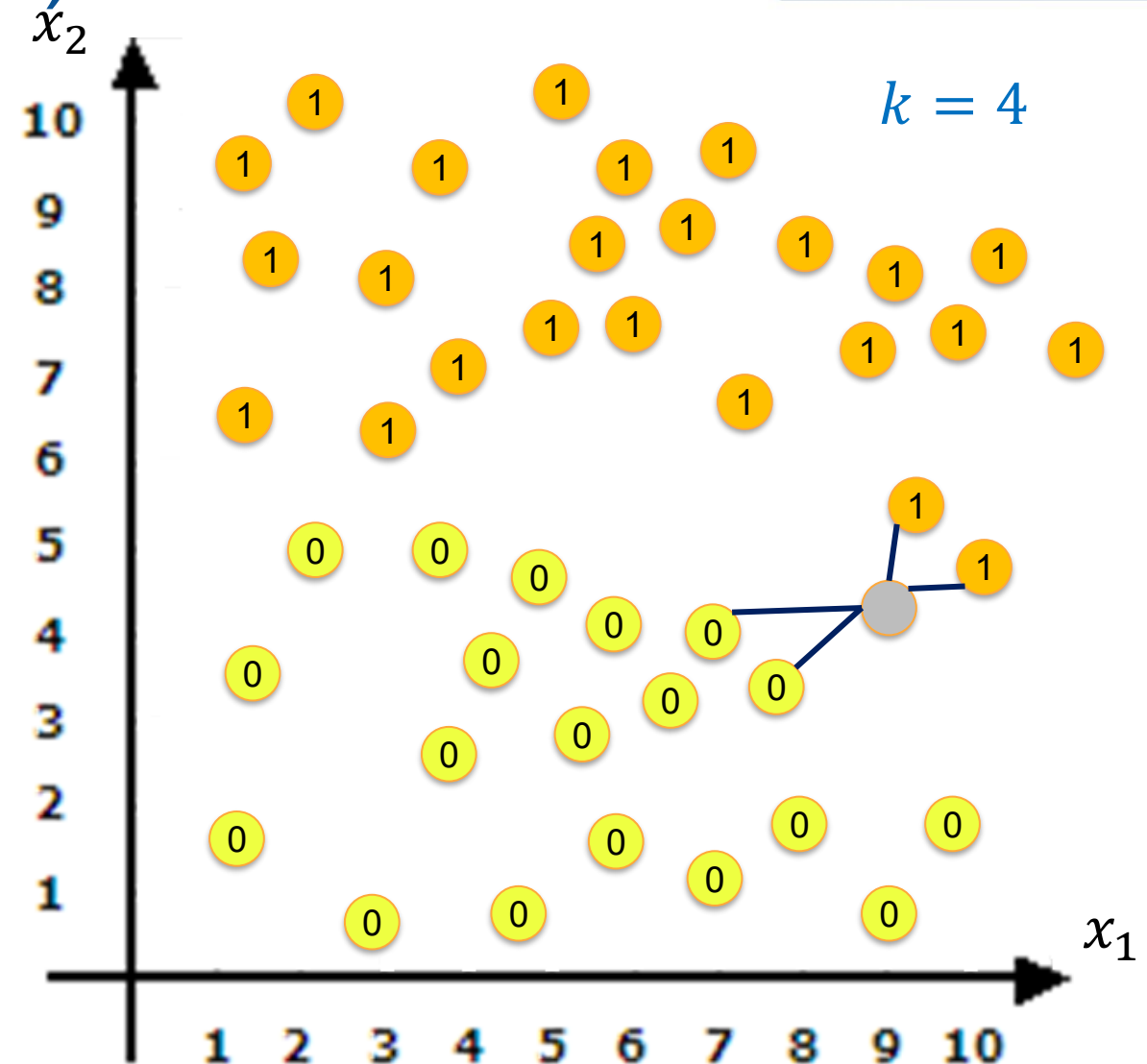
- Soft prediction:

$$\hat{p}(c|\mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} 1[y_i = c]$$



k-Nearest Neighbour (kNN) Classifier

- **Weighted kNN:** While taking majority among k neighbours, they can be weighed by the reciprocal of their distance $w_i = \frac{1}{d(x, x_i)}$
- k can be chosen through:
 - Trial and error by observing the error in prediction for different values of k

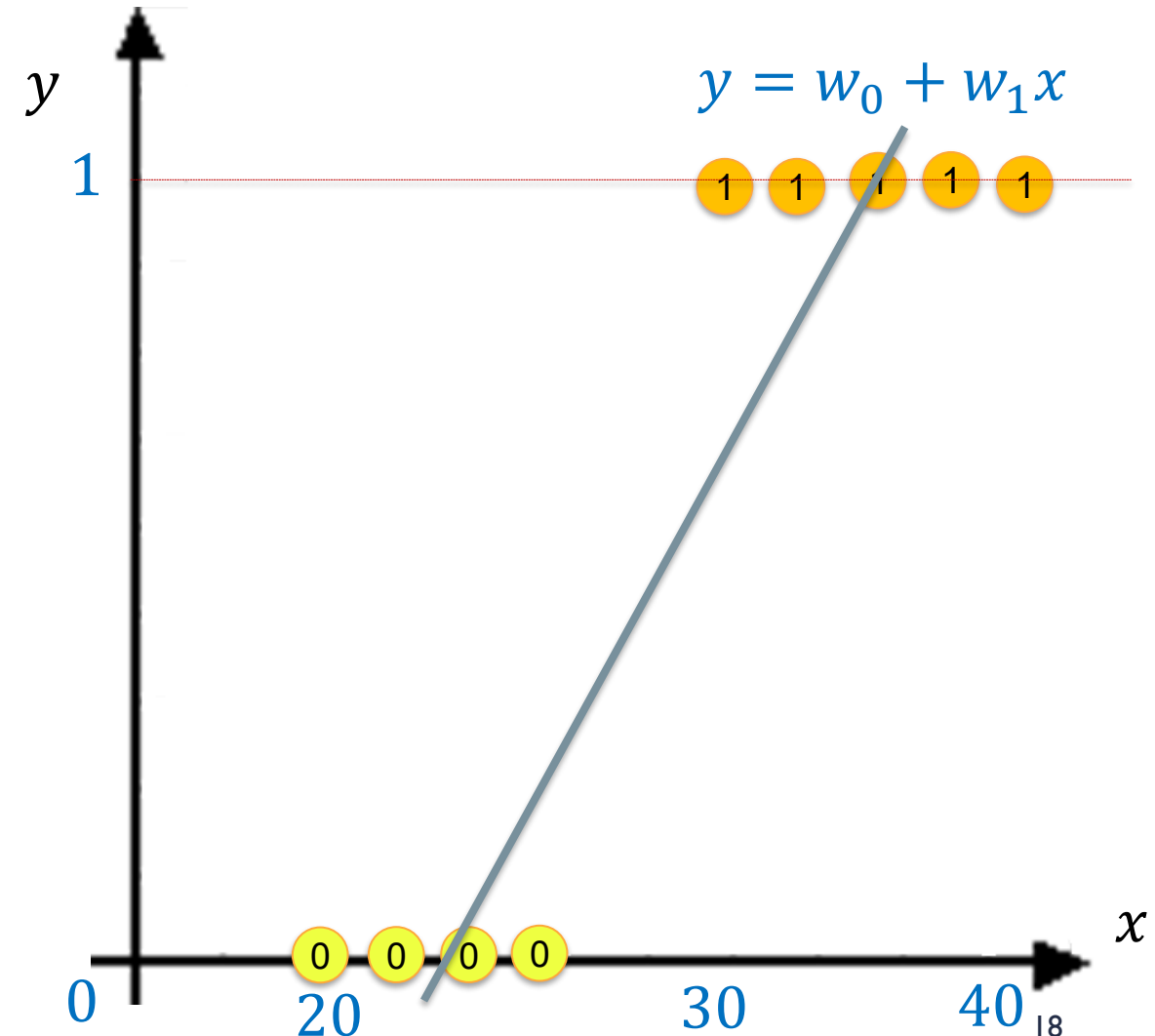


Logistic Regression

Logistic Regression - Motivation

- **Objective:** To make soft predictions i.e., probability of a sample belonging to a particular class
- **Question:** Can linear regression be used to predict classes (y) given x ?
- **Eg: Binary Classification** - To predict whether a person is normal or obese based on BMI
- Suppose linear regression is performed to predict y in this case
- **Issue:** Output is not guaranteed to lie between 0 and 1 and spans $(-\infty, \infty)$
- **Question:** Can the output of regression be transformed into a probability value?

x	20.5	23	24.6	25	31.5	33	35	38.6	40
y	0	0	0	0	1	1	1	1	1



Logistic Regression – Logit Link

- **Goal:** Model $p = P(y = 1|x)$
- **Idea:** Model the probability via function that always outputs $(0, 1)$
- Logit function transforms $(0, 1)$ to $(-\infty, \infty)$

$$\text{logit}(p) = \log \frac{p}{1-p}$$

- Therefore, logit function acts as a natural bridge between linear prediction (z) and probabilistic prediction (p)

$$\log \frac{p}{1-p} = z = w_0 + w_1 x$$

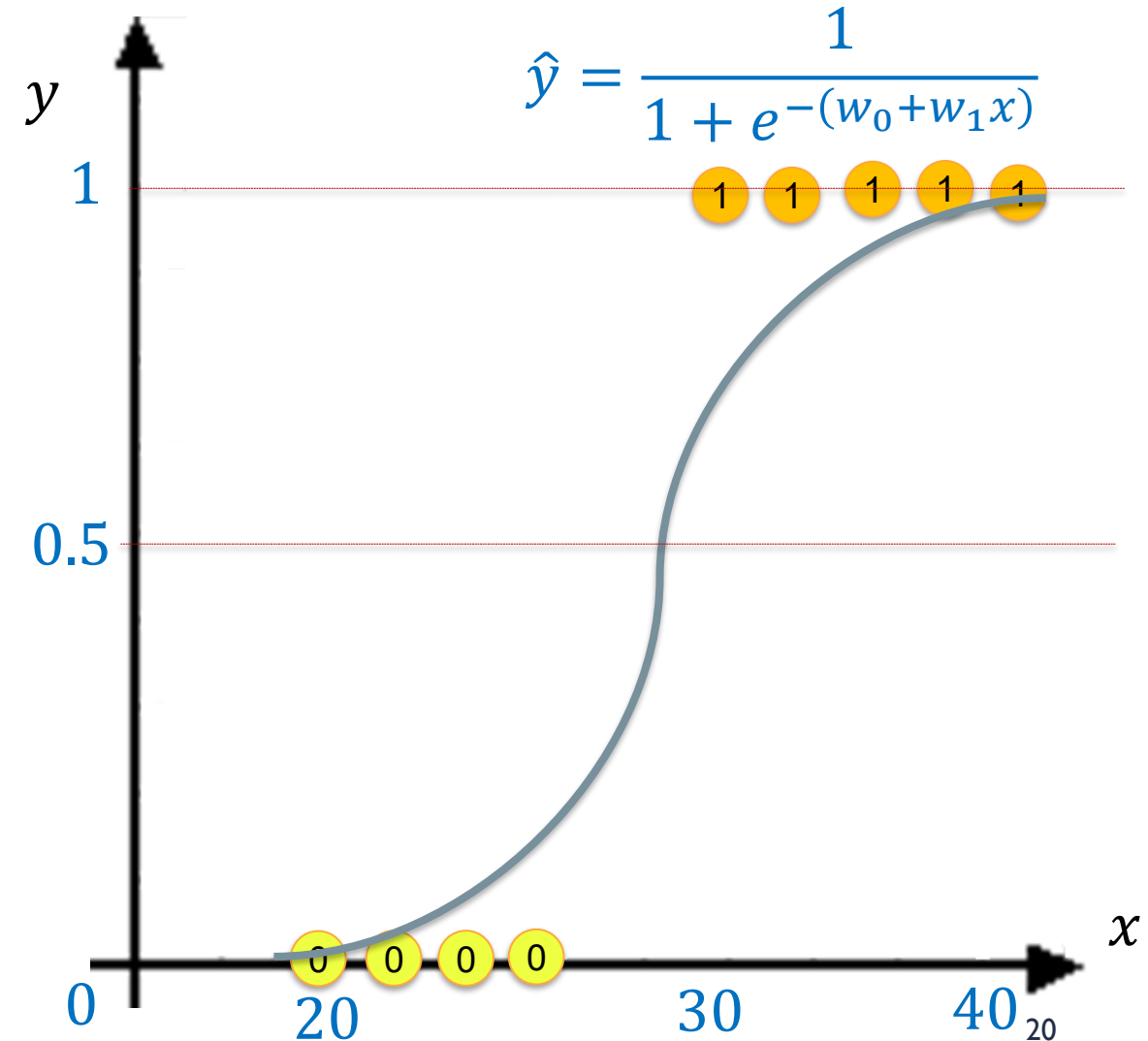
$$p = \frac{1}{1 + e^{-z}}$$

Logistic Regression – Sigmoid Function

- Sigmoid function maps any value (z) to a value between 0 and 1 (S-shaped)

$$p = \frac{1}{1 + e^{-z}}$$
$$\Pr(y = 1|x) = \frac{1}{1 + e^{-(w_0 + w_1 x)}}$$

- **Objective:** Parameters $w_0, w_1, \dots$ are to be obtained such that predicted y is close to actual y



Logistic Regression – Learning Parameters

- Known: Data, Model structure

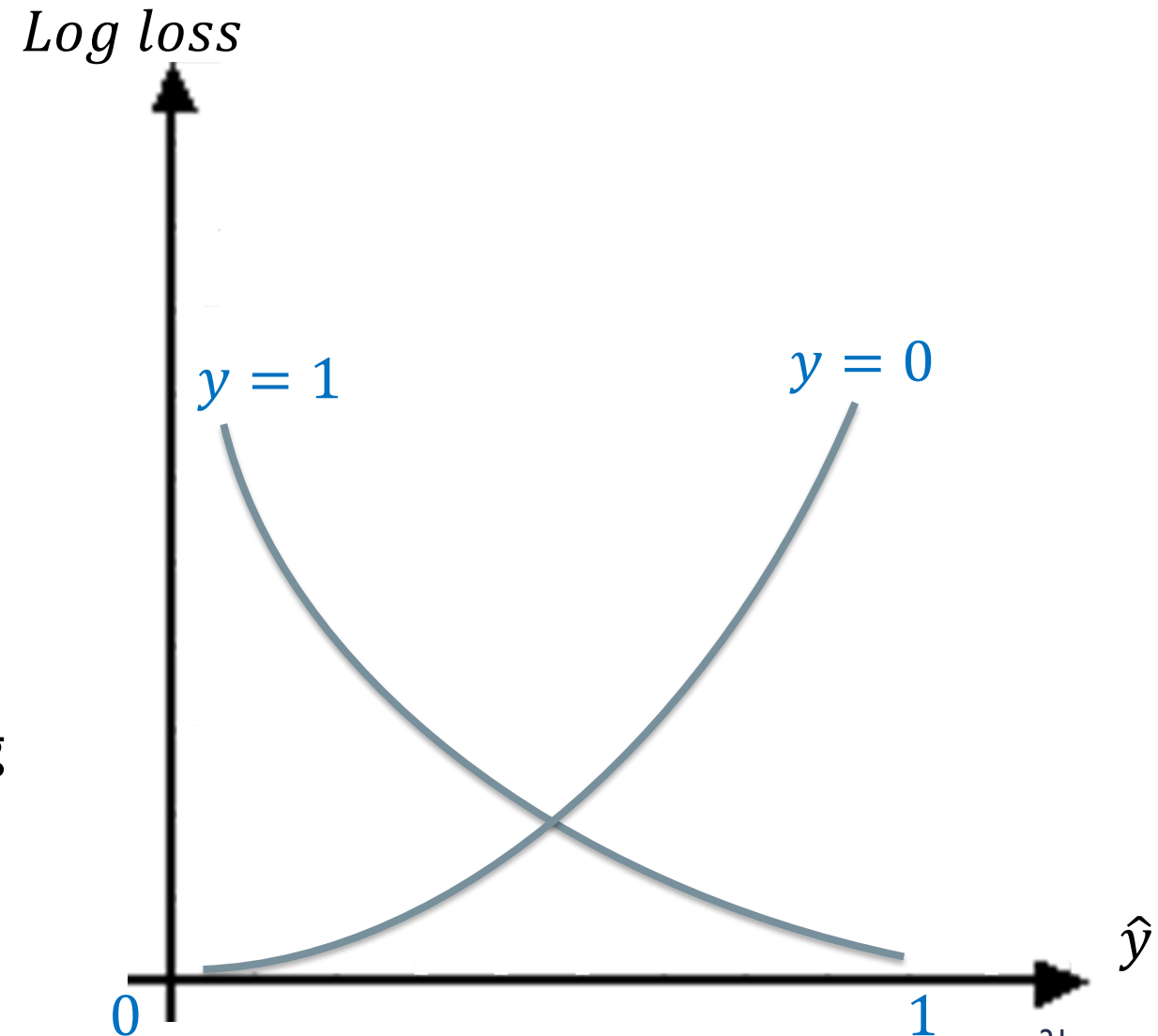
$$\hat{y} = \frac{1}{1 + e^{-(w_0 + w_1 x_1 + \dots + w_D x_D)}}$$

- Squared error loss cannot be used because it will have many local minima
- **Loss function:** Log loss is used because it goes to zero when prediction matches actual output

$$\begin{aligned} \log \text{ loss} &= -\ln \hat{y} \quad \text{if } y = 1 \\ &= -\ln(1 - \hat{y}) \quad \text{if } y = 0 \end{aligned}$$

$$l(y, \hat{y}) = -(y \ln \hat{y} + (1 - y) \ln(1 - \hat{y}))$$

- To obtain the parameters $w_0, w_1, \dots, w_D$ log loss is minimised over all samples using gradient descent technique



Logistic Regression – Gradient Descent

- Objective:

$$J(w) = \frac{1}{N} \sum_{i=1}^N -(y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i))$$

- Gradients:

$$\frac{\partial J}{\partial w} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i) x_i$$

$$\nabla_w J = \frac{1}{N} X^T (y - \hat{y})$$

- Gradient Descent update rule:

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial J}{\partial w}$$

Logistic Regression – Decision Boundary

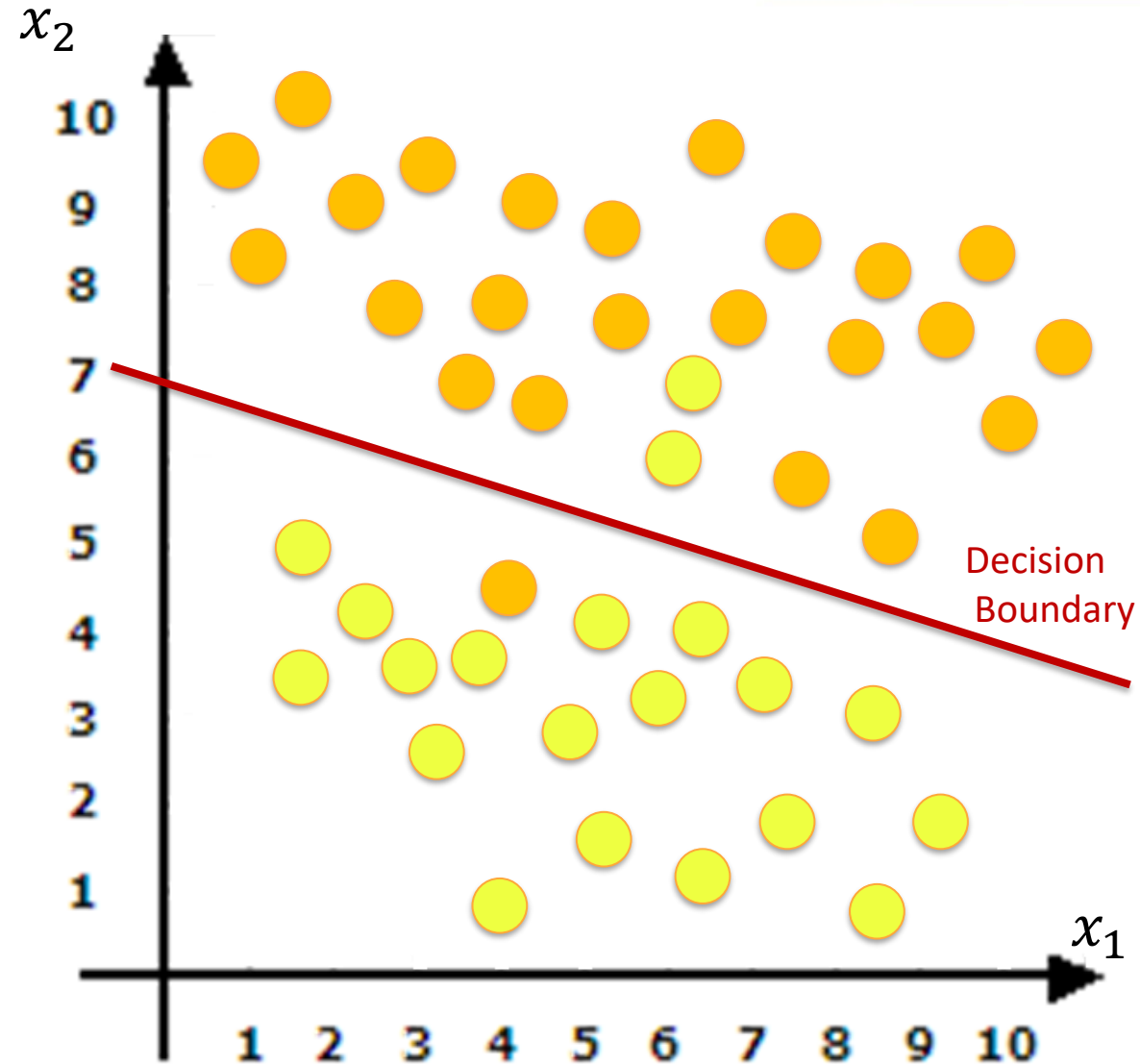
- To find out the decision boundary given by logistic regression, set the probability to threshold value (0.5)

$$\hat{y} = \frac{1}{1 + e^{-(w_0 + w_1x_1 + \dots + w_Dx_D)}} = 0.5$$

$$e^{-(w_0 + w_1x_1 + \dots + w_Dx_D)} = 1$$

$$w_0 + w_1x_1 + \dots + w_nx_n = 0$$

- **Interpretation:** Decision boundary is a line or a hyperplane which minimises the loss



Multiclass Logistic Regression

Multiclass Logistic Regression - Motivation

- **Objective:** To predict the probability that a data point belongs to each of the m classes
- All the m probabilities should add up to 1
- **Idea:** To use a separate set of parameters for each class
- Learn parameters $w_{0i}, \dots, w_{Di}$ for $i = 1, \dots, m$
- Rest of the steps from calculating loss to performing gradient descent remain the same
- **Question:** How to ensure that the sum of probabilities of all m classes adds up to 1 ?
 - **Replace Sigmoid with Softmax function**

p classes

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \end{bmatrix} \rightarrow \hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix}$$

$$\hat{y}_1 + \hat{y}_2 + \dots + \hat{y}_m = 1$$

$$z_{n,c} = w_{0i} + w_{1i}x_1 + \dots + w_{Di}x_D$$

where $i = 1, 2, \dots, m$

Softmax Function

- Sigmoid function gives a value between 0 and 1, and can be used for binary classification
- However, sigmoid cannot output multiple probability values which add upto 1 (multi-class)
- Softmax function is an extension of sigmoid function
- Softmax calculates the relative probabilities of multiples classes and ensures that total probability is 1

Input to softmax function

$$\mathbf{z} = [z_1 \quad z_2 \quad \dots \quad z_m]$$

Total Probability = 1

Prob. of class 0	Prob. of class 1
------------------	------------------

Binary Classification

Total Probability = 1

Prob. of class 1	Prob. of class 2	Prob. of class 3
------------------	------------------	------------------

Multi-class Classification

$$\hat{y}_{i,c} = \frac{e^{z_{i,c}}}{\sum_{j=1}^m e^{z_{i,j}}}$$

Cross Entropy Loss

- For classification problems with more than two classes, cross entropy loss is used
- Cross entropy loss: $\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^m y_{i,c} \ln \hat{y}_{i,c}$
- Consider a classification problem with output having 4 classes
- For each input, the probability of the input belonging to each of the four classes is predicted (softmax probability)
- Consider a sample belonging to class 1

$$\mathbf{y} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Desired output

$$\hat{\mathbf{y}} = \begin{bmatrix} 0.7 \\ 0.25 \\ 0.03 \\ 0.02 \end{bmatrix}$$

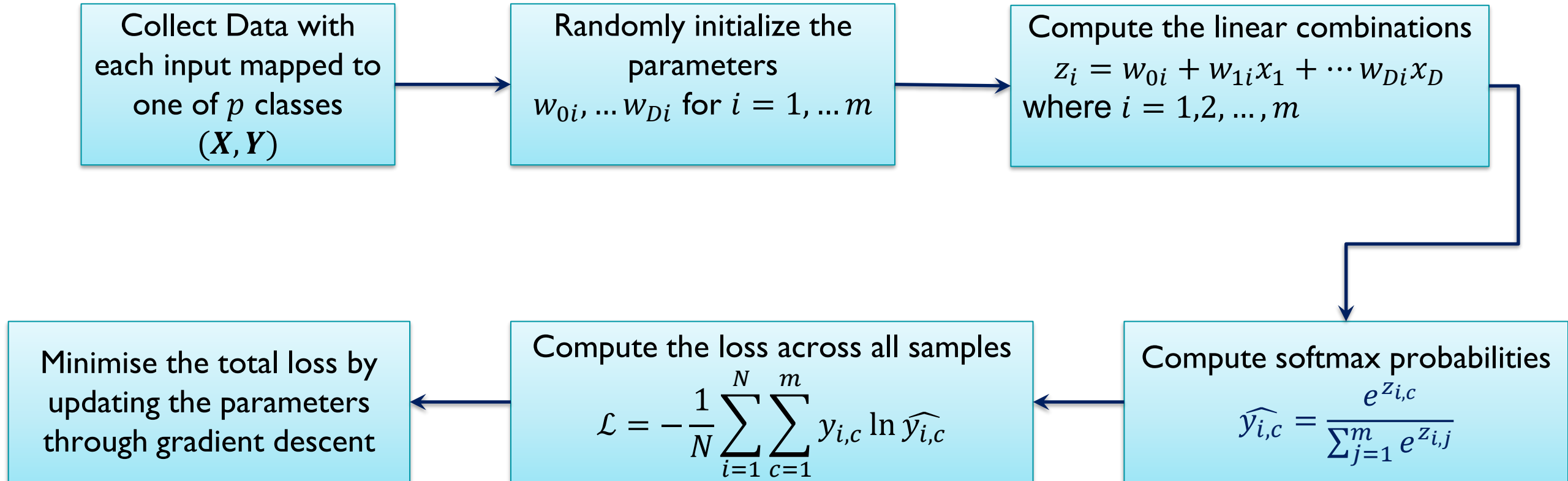
Predicted output

$$\mathcal{L} = -(1 * \ln(0.7) + 0 * \ln(0.25) + 0 * \ln(0.03) + 0 * \ln(0.02))$$

$$\mathcal{L} = -(-0.357)$$

$$\mathcal{L} = 0.357$$

Multiclass Logistic Regression - Summary



Performance Metrics for Classification

Errors in Classification

- Ideally, the classifier should predict the class labels of all the samples without any errors
- Since errors are inevitable, one needs to track for how many samples, the class labels are predicted correctly and for how many the predicted labels are erroneous – **Confusion Matrix**

Confusion Matrix

- A table or a matrix with different combinations of predicted and actual values for different classes for a given probability threshold
- For a binary classification task, the confusion matrix is:

		Actual Values	
		Positive (Class 1)	Negative (Class 0)
Predicted Values	Positive (Class 1)	True Positives (TP)	False Positives (FP)
	Negative (Class 0)	False Negatives (FN)	True Negatives (TN)

Confusion Matrix

		Actual Values	
		Positive (Class 1)	Negative (Class 0)
Predicted Values	Positive (Class 1)	True Positives (TP)	False Positives (FP)
	Negative (Class 0)	False Negatives (FN)	True Negatives (TN)

- **TP:** Sample Class is predicted to be positive (1) and its actual class is also positive (1)
- **TN:** Sample Class is predicted to be negative (0) and its actual class is also negative (0)
- **FP:** Sample Class is predicted to be positive (1) but its actual class is negative (0)
- **FN:** Sample Class is predicted to be negative (0) but its actual class is positive (1)

Building a Confusion Matrix: Example

Sample ID	Predicted Class	Actual Class
1	0	0
2	1	0
3	1	1
4	0	0
5	1	1
6	1	1
7	1	0
8	0	1
9	0	0
10	1	1

Predicted Values

Positive (Class 1)

Negative (Class 0)

Actual Values

Positive (Class 1)

Negative (Class 0)

4	2
1	3

Performance Metrics

- Different performance metrics for classifiers have been defined based on the values in the confusion matrix
 - Accuracy
 - Precision
 - Recall
- Other performance metrics have also been defined including:
 - F1 Score
 - Balanced Accuracy
 - ROC-AUC

Performance Metrics: Accuracy

- Simplest of performance metrics used in classification
- Suitable for both binary and multi-class classification

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

- Accuracy is 1 when FP and FN are zero
- Accuracy is a good choice for evaluating a classifier when the samples are well balanced and not skewed towards one class
- Accuracy is not useful when the dataset is largely imbalanced

Performance Metrics: Precision

- Indicates what proportion of the predicted positives are actually positive

$$Precision = \frac{TP}{TP + FP}$$

- Precision is 1 when there are no false positives
- Precision is a good choice for evaluating a classifier when cost of a false positive is high and cost of a false negative is low
- **Example:** Predicting whether an email is spam or not

Performance Metrics: Recall

- Indicates what proportion of actual positives are correctly classified i.e., calculates percentage of actual positives a model correctly identified

$$Recall = \frac{TP}{TP + FN}$$

- Recall is 1 when there are no false negatives
- Recall is a good choice for evaluating a classifier when cost of a false positive is low and cost of a false negative is high
- **Example:** Predicting whether a person has a disease or not

Performance Metrics: F1 Score

- F1 score is the harmonic mean of precision and recall which takes a value between 0 and 1

$$F1\ score = 2 * \frac{precision * recall}{precision + recall}$$

- F1 score maintains a balance between precision and recall i.e., give equal weightage to both
- Useful when we want a classifier model with both good precision and recall

Performance Metrics: Balanced Accuracy

- Balanced accuracy is the mean of sensitivity and specificity

$$\text{Sensitivity} = \text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Specificity} = \frac{TN}{TN + FP}$$

- In other word, sensitivity is the proportion of true positive among all positives
- Specificity is the proportion of false positives among all the negatives

$$\text{Balanced accuracy} = \frac{\text{Sensitivity} + \text{Specificity}}{2}$$

- Gives an accuracy values with appropriate weightage to both classes

F1 Score Vs Balanced Accuracy

- F1 score gives more importance to positives in the dataset
- F1 score is ideal with model performance is more associated to identifying positive class (Detecting fraud, disease, fault, etc)
- Balanced accuracy gives importance to both positives and negatives
- To be used when there is equal Importance for positives & negatives
- Balanced accuracy can be even used for balanced data while F1 score can mislead on balanced data

Imbalanced data

		Actual Values	
		Positive	Negative
Predicted Values	1	5	10
	0	5	980

Balanced accuracy = 0.745

F1 score = 0.4

Balanced data

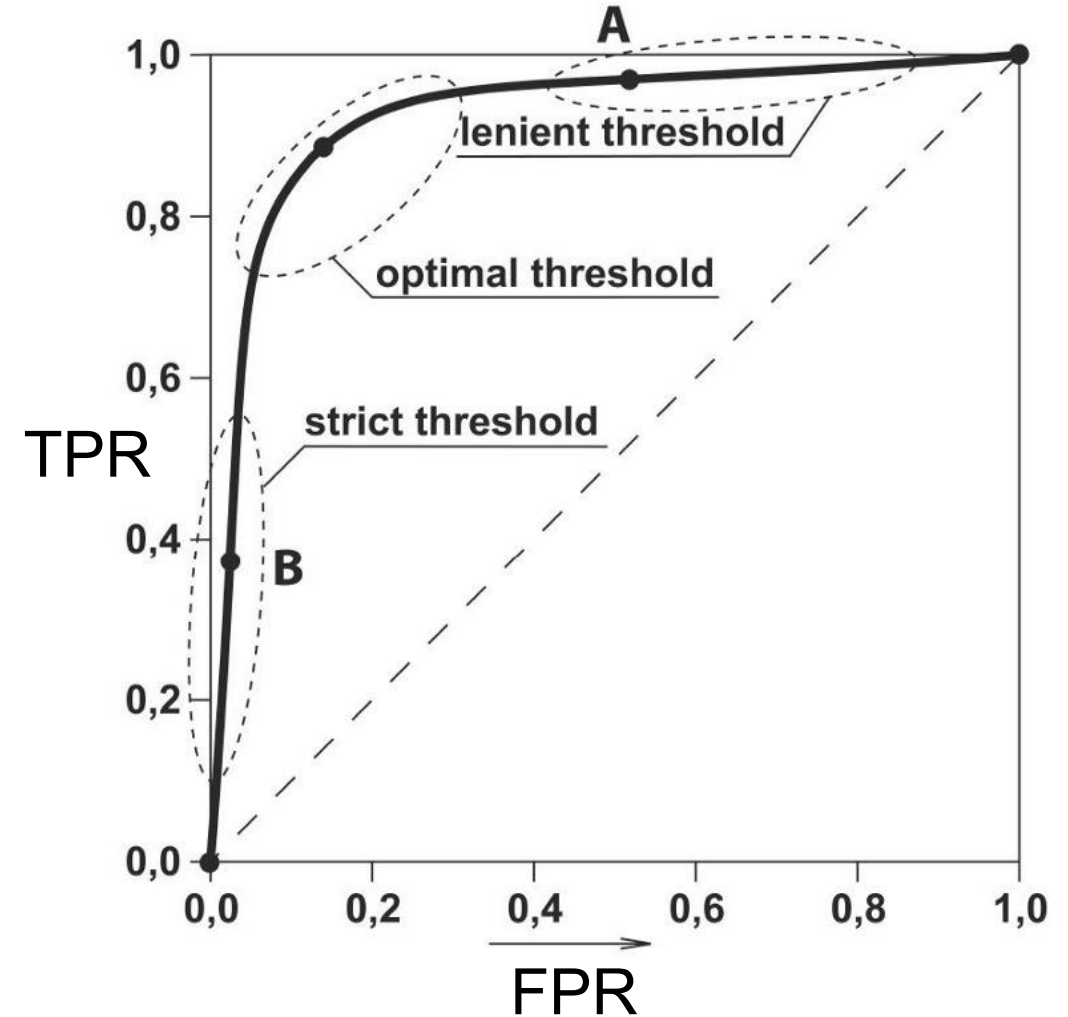
		Actual Values	
		Positive	Negative
Predicted Values	1	5	10
	0	5	0

Balanced accuracy
= 0.16

F1 score = 0.4₄₀

Performance Metrics: ROC-AUC

- ROC Curve is the plot of True Positive Rate Vs False Positive Rate for varying threshold
- $$TPR = \frac{TP}{TP + FN} ; FPR = \frac{FP}{FP + TN}$$
- ROC curve helps to visualise trade-offs between TPR and FPR
- Threshold generally depends on costs of FP and FN
- One option:** Maximise $J = TPR - FPR$
- ROC-AUC:** Area under the ROC curve gives a good measure of performance across thresholds
- Note:** Can be misleading with data is highly imbalanced



```
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
    operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
    operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
#selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
= ("Selected" + str(modifier_ob.name))  
mirror_ob.select = 0  
= bpy.context.selected_objects  
data.objects[one.name].select  
print("please select exactly one mirror")
```

WILLIAM C. LEE

```
def mirror(modifier):  
    #add mirror to the selected  
    #object -mirror_x  
    mirror_x
```

THANK YOU